

SkillWiz AI Learning Agent

AI Agent Development Assignment – Submission

Assignment	AI Agent Development – SkillWiz
Stack	Python · LangChain · Claude (Anthropic) · SQLite
Tools Implemented	fetch_student_progress · suggest_next_topic
Memory	LangChain ConversationBufferMemory
Database	SQLite (students.db)

1. Project Overview

The SkillWiz AI Agent is a conversational learning assistant that helps students decide what to study next based on their personal progress data. The agent integrates a SQLite database of student records, two custom LangChain tools, and a Claude LLM backend to produce context-aware, actionable study recommendations. It supports multi-turn conversation through LangChain's ConversationBufferMemory, ensuring it remembers context within a session.

The system satisfies all assignment requirements: it fetches real student data, reasons about the student's current phase and pending topics, and replies meaningfully to queries like *'What should I do today?'*

2. Architecture

The agent follows a Tool-Augmented LLM pattern. The LLM (Claude Sonnet) acts as the reasoning engine; it may call one or both tools to retrieve data before composing its reply.

Component	Role
SQLite Database	Persistent store for student progress records
Tool 1 – fetch_student_progress	Queries the DB and returns a student's full record
Tool 2 – suggest_next_topic	Analyses progress and returns a prioritised suggestion
LangChain Agent (ZERO_SHOT_REACT)	Orchestrates tool use and LLM reasoning
ConversationBufferMemory	Maintains turn-by-turn context in a session
Claude Sonnet (Anthropic)	Generates natural-language responses

3. Project Structure

```
skillwiz-agent/
■■■ agent.py          # Main agent definition and entry point
■■■ tools.py          # Custom LangChain tools
■■■ database.py       # SQLite setup and helper functions
■■■ students.db       # Auto-generated SQLite database
■■■ requirements.txt  # Python dependencies
■■■ README.md         # Setup and usage instructions
```

4. Full Source Code

4.1 database.py

```
import sqlite3, json

DB_PATH = "students.db"

def init_db():
    """Create the students table and seed sample data."""
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS students (
            user_id          TEXT PRIMARY KEY,
            target            TEXT,
            current_phase     TEXT,
            completed_topics TEXT,    -- JSON array
            pending_topics   TEXT,    -- JSON array
            streak            INTEGER
        )
    """)
    # Seed sample student
    cur.execute("""
        INSERT OR IGNORE INTO students VALUES (
            'user_123', 'Frontend Developer', 'JavaScript Basics',
            '["Variables", "Loops"]',
            '["Promises", "Async/Await", "ES6 Modules", "DOM Manipulation"]',
            5
        )
    """)
    conn.commit()
    conn.close()

def get_student(user_id: str) -> dict | None:
    """Return a student record as a dict, or None if not found."""
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    cur = conn.cursor()
    cur.execute("SELECT * FROM students WHERE user_id = ?", (user_id,))
    row = cur.fetchone()
```

```

conn.close()
if not row:
    return None
data = dict(row)
data["completed_topics"] = json.loads(data["completed_topics"])
data["pending_topics"] = json.loads(data["pending_topics"])
return data

```

4.2 tools.py

```

from langchain.tools import tool
from database import get_student

@tool
def fetch_student_progress(user_id: str) -> str:
    """
    Fetch a student's learning progress from the database.
    Input: user_id (string)
    Returns a formatted summary of the student's current state.
    """
    student = get_student(user_id)
    if not student:
        return f"No student found with user_id '{user_id}'."

    completed = ", ".join(student["completed_topics"]) or "None"
    pending = ", ".join(student["pending_topics"]) or "None"

    return (
        f"Student ID : {student['user_id']}\n"
        f"Goal : {student['target']}\n"
        f"Current Phase : {student['current_phase']}\n"
        f"Completed : {completed}\n"
        f"Pending : {pending}\n"
        f"Study Streak : {student['streak']} day(s)"
    )

@tool
def suggest_next_topic(user_id: str) -> str:
    """
    Suggest the most appropriate next learning action for a student.
    Input: user_id (string)
    Returns a specific, context-aware recommendation.
    """
    student = get_student(user_id)
    if not student:
        return f"No student found with user_id '{user_id}'."

    pending = student["pending_topics"]
    completed = student["completed_topics"]
    streak = student["streak"]
    phase = student["current_phase"]
    total = len(pending) + len(completed)

```

```
# Decide recommendation type
if not pending:
    suggestion = (
        f"You've completed all topics in '{phase}'! "
        "Consider taking a project challenge to consolidate your skills, "
        "or move on to the next phase."
    )
elif len(completed) / total >= 0.7:
    suggestion = (
        f"Great progress in '{phase}'! Next recommended topic: "
        f"'{pending[0]}'. After that, consider a mini-project to "
        "apply everything you've learned so far."
    )
else:
    suggestion = (
        f"Keep going in '{phase}'! Focus on: '{pending[0]}'. "
        f"You have {len(pending)} topic(s) remaining. "
        + (f"Your {streak}-day streak is impressive – keep it up!"
           if streak >= 3 else "Try to study daily to build your streak!")
    )
return suggestion
```

4.3 agent.py

```
import os
from langchain_anthropic import ChatAnthropic
from langchain.agents import initialize_agent, AgentType
from langchain.memory import ConversationBufferMemory
from database import init_db
from tools import fetch_student_progress, suggest_next_topic
```

[illegible]

```
llm = ChatAnthropic(
    model="claude-sonnet-4-20250514",
    anthropic_api_key=os.environ["ANTHROPIC_API_KEY"],
    temperature=0.3,
)
```

```
tools = [fetch_student_progress, suggest_next_topic]
```

```
memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True,
)
```

```
SYSTEM_PROMPT = """You are SkillWiz, a friendly and motivating AI learning coach.
You have access to a student's learning database through your tools.
Always fetch the student's progress before making suggestions.
Give specific, encouraging advice tailored to their exact situation.
Be concise but warm. The default student ID is 'user_123'."""
```

```
agent = initialize_agent(
```

```
tools=tools,
llm=llm,
agent=AgentType.CHAT_CONVERSATIONAL_REACT_DESCRIPTION,
memory=memory,
verbose=True,
agent_kwargs={"system_message": SYSTEM_PROMPT},
)
```

```
# ■■■ Main loop ■■■  
def chat(user_input: str) -> str:  
    return agent.run(user_input)  
  
if __name__ == "__main__":  
    print("SkillWiz Agent ready. Type 'quit' to exit.\n")  
    while True:  
        query = input("You: ").strip()  
        if query.lower() in ("quit", "exit"):  
            break  
        if not query:  
            continue  
        response = chat(query)  
        print(f"\nSkillWiz: {response}\n")
```

4.4 requirements.txt

```
langchain>=0.2.0
langchain-anthropic>=0.1.0
anthropic>=0.28.0
```

5. Sample Agent Interaction

Below is a representative console session demonstrating the agent's contextual reasoning across multiple turns. The agent automatically calls tools before responding.

```
$ python agent.py
SkillWiz Agent ready. Type 'quit' to exit.
```

You: What should I do today?

```
> Entering new AgentExecutor chain...
```

Action: `fetch_student_progress`

Action Input: user_123

Observation:

Student ID : user_123

Goal : Frontend Developer

Current Phase : JavaScript Basics

Completed : Variables, Loops

Pending : Promises, Async/Await, ES6 Modules, DOM Manipulation

Study Streak : 5 day(s)

Action: suggest_next_topic

```
Action Input: user 123
```

Observation: Keep going in 'JavaScript Basics'! Focus on: 'Promises'.
You have 4 topic(s) remaining. Your 5-day streak is impressive – keep it up!

SkillWiz: Great work keeping your 5-day streak alive! Today I'd recommend diving into **Promises** – it's the foundation for all asynchronous JavaScript. Try building a small fetch-based app once you're comfortable with the syntax. You're 33% through JavaScript Basics. Keep going!

You: I already finished Promises last night.

SkillWiz: That's fantastic progress! Since you've knocked out Promises, your next focus should be **Async/Await** – it's essentially syntactic sugar on top of Promises, so the concepts will feel very natural after what you just learned. Aim for 45 minutes today and you'll be well on your way.

6. README – Setup Instructions

Step 1: Clone the repository

```
git clone https://github.com/lavishsaini136-rgb/skillwiz-agent.git
cd skillwiz-agent
```

Step 2: Create & activate a virtual environment

```
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
```

Step 3: Install dependencies

```
pip install -r requirements.txt
```

Step 4: Set your Anthropic API key

```
export ANTHROPIC_API_KEY=sk-ant-... # Windows: set ANTHROPIC_API_KEY=sk-ant-...
```

Step 5: Run the agent

```
python agent.py
```

The SQLite database (students.db) is created automatically on first run. No external database server is required.

7. Requirements Coverage

Requirement	How It Is Fulfilled
Fetch student data from a DB	fetch_student_progress tool queries SQLite
Understand current phase/status	Tool returns phase, completed & pending topics
Suggest next step	suggest_next_topic applies branching logic for advice

Handle 'What should I do today?'	Demonstrated in sample interaction (Section 5)
Use an LLM	Claude Sonnet via langchain-anthropic
LangChain (preferred)	Agent, tools, and memory all use LangChain
At least 2 tools	fetch_student_progress + suggest_next_topic
Basic memory/context handling	ConversationBufferMemory across turns
Database	SQLite — zero config, file-based, seeded automatically
Contextual (not generic) output	Branching logic in tool + Claude reasoning

8. Design Decisions & Notes

- **SQLite over MongoDB/JSON:** SQLite requires zero setup, runs entirely in-process, and satisfies the assignment's database requirement without additional infrastructure.
- **Two distinct tools:** Separating data-fetching (fetch_student_progress) from recommendation logic (suggest_next_topic) keeps each tool single-responsibility and makes the agent's reasoning chain transparent in verbose output.
- **Claude as the LLM:** Claude Sonnet is used via the official langchain-anthropic integration. Any OpenAI-compatible LLM can be swapped in by changing the llm= line.
- **ConversationBufferMemory:** Chosen for simplicity; it holds the full chat history in memory for the session, enabling natural follow-up questions without re-fetching data.
- **Branching recommendation logic:** The suggest_next_topic tool applies explicit rules (all done → project challenge; 70%+ done → mini-project; otherwise → next topic) so the LLM receives a concrete starting point rather than raw data, reducing hallucinations.

9. Submission Links

GitHub Repository: <https://github.com/lavishsaini136-rgb/skillwiz-agent>

Demo Video: Record a 2–3 minute Loom or screen-capture showing the agent running in the terminal with at least two conversational turns. Upload the link to the GitHub README.